Refinement-based verification of cryptographic primitives

Fernando Leal

August 22, 2025
M1 internship report
Prosecco team, Inria Paris, supervised by Aymeric Fromherz and Son Ho

Contents

1. Introduction

In 2022, cryptography researcher Nicky Mouha published CVE-2022-37454 [1], which exposed a vulnerability in the implementation of the SHA3 hash function found in the e**X**tended **K**eccak **C**ode **P**ackage[1], henceforth referred to as XKCP. This vulnerability allowed malicious actors to violate cryptographic properties of the hash function and potentially achieve arbitrary code execution. The vulnerability impacted a number of downstream projects using the XKCP's implementation, including Python's `hashlib` for versions prior to 3.10, PHP and the `SHA3 for Ruby` gem. This incident serves to show that correctly implementing cryptographic primitives, such as SHA3, is a hard endeavour. The vulnerability had been present in the code for 11 years before being discovered. One of the reasons it took so long to be spotted was how specific the conditions for the vulnerability to be triggered were, and how unlikely it was to hit those same conditions in regular use of the code.

Cryptographic primitives are defined in specifications, which are publications released in the form of standards by organizations such as IEEE[2] or NIST[3]. These specifications describe how a proper implementation of the cryptographic primitive should behave under all inputs. However, implementations regularly employed in real-world applications differ greatly in appearance from their specifications, usually due to optimizations or translations to a more constrained target programming language. These differences in appearance allow for discrepancies between the implementation and the specification to more easily remain undetected, more so if they are only present under specific conditions on their inputs. Unfortunately, even if regular users are unlikely to be impacted by these discrepancies directly, this does not prevent malicious actors from exploiting them so it is crucial to ensure that none of these discrepancies exist.

To address these difficulties, projects like Fiat-Crypto [2] and HACL* [3] have opted to provide proofs of correctness for their implementations of cryptographic primitives. These proofs of correctness are meant to assert that the implementations behave as specified in their respective standards, and are carried out in proof assistants, such as Rocq and F*, which ensure their validity mathematically.

Proof assistants are not usually able to refer directly to the implementation's code in their proofs, which means that proof engineers must translate their implementation into their proof assistant of choice for them to be able to prove things about it. This leads to a duplicated definition of the implementation, one for the final artifact and another for the proof assistant. This duplication of the information is tedious and error-prone, and proofs only work under the assumption that both representations are indeed equivalent. To address this issue, projects like HACL* have opted to generate the final artifact of the implementation from its description in the proof assistant, by compiling F* code to efficient C code [4]. Other tools, such as Aeneas [5], take the inverse approach, by automating the translation of the final artifact's code into descriptions for various proof assistants, such as Rocq, F* or Lean. With this approach, only the tool needs to be trusted, and not each individual translation.

Likewise, specifications of cryptographic primitives must also be translated into the proof assistant of choice for them to be able to refer to it. The standards in which these specifications are found describe them using natural language and pseudocode, which do not lend themselves as well to automatic translation since lack precise formal semantics. Therefore, a formal specification can only be written by a proof engineer, trying to follow the specified standard as closely as possible. This translation is not always straight-forward, since the chosen proof assistant may introduce restrictions in how it can be described, requiring the proof engineer to adapt the definitions and risking to differ from the standard in doing so. This step remains crucial though, since the proofs of correctness are only valid as long as the formal specification they refer to also is.

[1] https://github.com/XKCP/XKCP

[2] https://www.ieee.org/

[3] https://www.nist.gov/

In my internship, I explored the methodology of refinement-based verification for cryptographic implementations, from the translation of the standard into a formal specification to the development of a verified implementation, and proof engineering techniques to manage the complexity found in these proofs. To do so, I used the Aeneas verification toolchain to translate a SHA3 implementation written in Rust to definitions in the Lean4 [6] proof assistant, and prove they implement the SHA3 specification correctly for all inputs, and particularly that the Rust code does not panic.

2. Case study: SHA3

Our main case study is SHA3, a family of hashing functions standardized by NIST. We wrote a SHA3 implementation in Rust, a modern programming language leveraging a rich type system to produce safe and extremely performant executables, and verified it using Lean, a modern and extensible theorem prover built upon the Calculus of Inductive Constructions [7].

The formal SHA3 specification used for the proofs of correctness, also developed in this project, can be found in GitHub under the repository `sha3.lean`[4] as a standalone Lean library. The code of the implementation can be found in the repository `sha3.rs`[5], which defines the crate `shars`. The same repository also includes the proofs of correctness of the implementation, as well as the Lean definitions generated by Aeneas from the Rust implementation. The development of `shars` was divided in two stages.

Stage one. In the first stage, an implementation was developed to follow as closely as possible the formal specification. The motivation behind this decision was to arrive at a working verified implementation in Rust with the least amount of complexity in the proofs as possible. To this end, efficiency of the code was sacrificed to use a representation as close as possible to the one found in the specification. For instance, since the SHA3 specification handles lists of bits, in Rust we represented these as slices of booleans `&[bool]`.

Stage two. In the second stage, we built up from the implementation in the first stage to a more reasonable implementation, with some applied optimizations and a more efficient representation of lists of bits as machine integers. Our aim in the second stage was to be able to focus on the changes introduced by optimizations and new representation while reusing as much as possible the proofs developed in the previous stage.

SHA3 overview. SHA3 is a cryptographic hashing function, and is composed of two main parts: a permutation function and a sponge construction. The permutation function used in SHA3 is the Keccak-f[1600] function, which is itself defined as the repeated application of certain step functions θ, ρ, π, χ and ι. The sponge construction takes this permutation function and uses it to implement a function which maps an arbitrary number of bits to another arbitrary number of bits. It does so through two steps. In the *absorb* step, the user input is gradually fed to the permutation function to derive a state vector. In the *squeeze* step, the previously derived state vector is used to generate an arbitrary number of bits as an output. The steps *absorb* and *squeeze* are where the sponge construction derives its name.

[4] https://github.com/ayhon/sha3.lean
[5] https://github.com/ayhon/sha3.rs

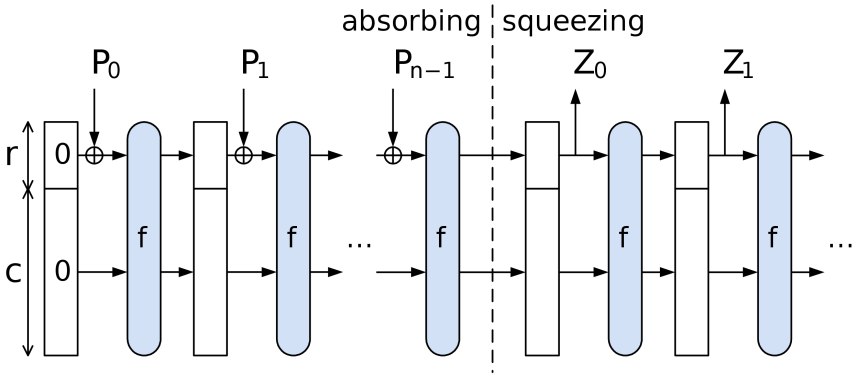
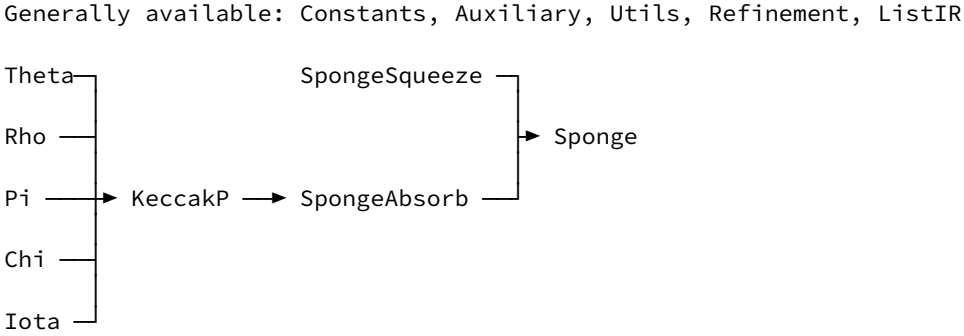


Figure 1: The sponge construction. $P_0 \parallel \ldots \parallel P_{n-1}$ is the input, $Z_0 \parallel \ldots \parallel Z_m$ is the output.

Proof architecture. Proving the correctness of these different components is the core of the verification effort of `shars`. Each proof of correctness for the previous functions took place in its own file, while theorems which are shared between different files but do not depend on the Rust models are found in the `Utils` module. Structuring the proofs in this way ensures that changes made to some part of the proofs only affect the parts of the proof that depend on them. While working on a proof, auxiliary definitions were included in the file where they were needed, and after all proofs were done, propagated to the file in which they should be stored (usually under the `Utils` folder).



Listing 1: Diagram of `shars`

3. Formal specifications, and how to trust them.

In the introduction we discussed the importance of a correct and trustworthy formal specification. The formal specification will determine what guarantees we are able to assert from our implementation. There are multiple methods that can be applied to earn this trust, several of which are explored in the current section.

Stay close to the standard. One of the main difficulties we identified when translating a cryptographic standard into a theorem prover as a formal specification was in the language used in the standard to describe its cryptographic primitive. Since the main audience of a cryptographic standard are implementors of said cryptographic primitive, the algorithms are described in a procedural style, with constructs such as `for` loops, mutable variables or `goto` statements . On the other hand, theorem provers usually prefer a functional description, which translates more directly to an underlying functional representation. When these procedural constructs are not supported by the theorem prover, it becomes the proof engineer's responsibility to perform this translation between procedural and functional.

In our case, the Lean proof assistant comes with support for mutable variables and `for` loops through the use of syntactic sugar in its `do` notation [8]. These constructs are later translated into an equivalent, but functional representation. Therefore, proof engineers are able to define the formal specification such that it mimics closely the standard, yet it benefits from the underlying functional representation when doing

proofs. As an example, compare ρ algorithm's description in the SHA3 standard in Figure 2 and its formal specification in the `sha3.lean`[6] repository in Listing 2.

Algorithm 2: $\rho(\boldsymbol{A})$
Input: state array $\boldsymbol{A}$ *Output:* state array $\boldsymbol{A}'$

1. For all z such that $0 \leq z < w$, let $\boldsymbol{A}'[0,0,z] = \boldsymbol{A}[0,0,z]$
2. Let $(x,y) = (1,0)$
3. For t from 0 to 23:
   a. for all z such that $0 \leq z < w$, let $\boldsymbol{A}'[x,y,z] = \boldsymbol{A}\left[x,y,\left(z - \frac{(t+1)(t+2)}{2} \bmod w\right)\right]$
   b. let $(x,y) = (y, (2x+3y) \bmod 5)$
4. Return $\boldsymbol{A}'$

Figure 2: Standard's description of the ρ function

```
def ρ(A: StateArray l): StateArray l :=
  Id.run do
  let mut A' := A
  let mut (x, y): Fin 5 × Fin 5 := (1,0)
  for t in List.finRange 24 do
    for z in List.finRange (w l) do
      A' := A'.set x y z <|
        A.get x y (z - ρ.offset t)
    (x, y) := (y, 2*x + 3*y)
  return A'
```

Listing 2: Formal specification of the ρ step function

Another Lean feature which allow us to stay as close as possible to the standard has to do with identifiers. Lean supports a large number of unicode characters in its identifiers, which we make use of to maintain the names of each algorithm defined in the standard, as it is the case for ρ in Listing 2. Furthermore, Lean allows an arbitrary unicode string to be used as an identifier as long as it is enclosed between «». Strings like «`pad10*1`» will be parsed as identifiers, instead of as an arithmetic expression.

Make your specification executable. Having the formal specification be executable means that the specification itself becomes an implementation of the standard. This is useful since the organizations publishing these standards usually provide a way to validate its implementations[7]. While these methods do not guarantee the correctness of the implementations, they do allow us to increase our confidence in them. Furthermore, we can also use other more mature implementations to test our formal specification against, which will allow us to find potential errors.

Making a Lean specification executable does not usually require any extra effort. Lean is able to prove the termination of most functions automatically, since this is a requirement for Lean's logic to be consistent. When termination is not deduced automatically, Lean can be made to accept the definition either by marking it with the `partial_fixpoint` keyword or by proving to Lean its termination. This latter option is done by associating a well-ordered value with each function call, and showing that recursive calls result in gradually decreasing associated values. In Listing 3, the `termination_by` keyword is used to associate each call of the function with the value `d - m`, with which Lean is then able to prove that the function is in fact terminating. The user can also give an explicit proof through the `decreasing_by` keyword, if Lean is unable to produce it automatically.

```
def sponge.squeeze
    (f: Bitstring (b l) → Bitstring (b l))
    (r: Nat)
    (Z: Bitstring m)
    (S: Bitstring (b l))
    (r_bnd: 0 < r ∧ r < b l := by omega)
: Bitstring d
:=
  if d <= m then
    Z.setWidth d
  else
    sponge.squeeze f r (Z ++ S.setWidth r) (f S)
termination_by d - m -- ← Termination hint
```

Listing 3: `sponge.squeeze` formal specification

[6] https://github.com/ayhon/sha3.lean
[7] https://csrc.nist.gov/projects/cryptographic-algorithm-validation-program

Finally, since our specification is itself an implementation, it is trivially a verified implementation against itself. This can be useful when developing optimizations that take into account precomputed values. If a particular value computation does not depend on user-provided values, it can be instead replaced with the value it computes. And to compute said value we can use our specification itself. For instance, in the SHA3 specification, the ι function makes use of a special *round constant* whose computation is non-trivial, but does not depend on user input, and can therefore be precomputed. Obtaining the values of the *round constant* can be trivially done in Lean with the code in Listing 4.

```
#eval Array.range 24 -- 24 rounds
  |>.map (Spec.Keccak.ι.RC (l := 6))
  |>.map (·.foldr (·.toNat + ·*2) 0)
```

Listing 4: Lean code to generate the values of ι's *round constant*.

Use dependent types. Since cryptographic standards are meant to guide engineers in implementing the specified algorithm, they often leave out details of the underlying mathematics of the algorithm itself. For instance, it is rare to see a specification make mention of safe array accesses, specific properties that hold over some values, or types other than numbers and lists of bits. However, thanks to dependent types we are able to capture some of these properties in the specification itself.

A perfect example of this is our pervasive use of the `Fin n` type. An element of type `Fin n` is a number `x` with the added constraint that `x < n`. In addition, the usual arithmetic operations are defined for `Fin n`, such as addition, subtraction or multiplication. To ensure that elements in `Fin n` maintain their bound with `n`, these operations are implemented modulo `n`.

By default, Lean does not allow arbitrary indexing of a list or array, and requires the user to provide a proof that the access is safe if it is not able to deduce it on its own[8], like in the case of Listing 5. However, indices `i` of type `Fin n` are automatically accepted by Lean to prove safe access of vectors of length `n`. This is used extensively in the specification to ensure that all accesses are safe. Furthermore, indices in the SHA3 specification are also only manipulated using modular operations. Having indices be represented as `Fin n` ensures that all of these operations are performed correctly by definition, and is also the reason why in Listing 2 there is no need to include modular operations in the update of `x` and `y`.

```
def ι.RC {l: Fin 7}(iᵣ: Nat): Bitstring (w l) := Id.run do
  let mut RC := .replicate (w l) false
  for j in List.finRange (l.val + 1) do -- inclusive range!
    have j_valid_idx := calc  2 ^ ↑j - 1
        _ < 2 ^ ↑j := Nat.pred_lt (Nat.ne_of_gt (Nat.two_pow_pos j.val))
        _ ≤ 2 ^ ↑l := Nat.pow_le_pow_iff_right (by decide) |>.mpr <| Fin.is_le j
    RC := RC.set (2^j.val - 1) (ι.rc (↑j + 7*iᵣ)) j_valid_idx
  RC      --                                      ↑ Direct proof of safe access
```

Listing 5: Computation of ι's *round constant*

4. Using refinements to handle information overload

One of the main obstacles when developing a verified implementation comes from the complexity of the proofs of correctness. When high-level specifications are compared with low-level descriptions of their implementations, the amount of definitions can be overwhelming. This is a similar issue to that found when developing big computer systems in software engineering, and therefore we apply the same solution. This is the principle of *separation of concerns* [9]. In the case of this project, we interpret each *concern* to be a particular aspect of the proof, like the termination of an algorithm or the correctness of a particular optimization. To construct our proofs of correctness, we structure them as a series of refinements.

We say that a definition is a *refinement* of another if, under some conditions, the former definition is equivalent to the latter. A refinement can be understood as a kind of subset relation which states that one representation is able to express everything the other can. As an example, we can consider `Nat` to be a

[8]Lean tries to prove safe accesses automatically with the extensible `get_elem_tactic`

refinement of `Fin n`. Any operation performed on elements of type `Fin n` can be translated to an operation on `Nat`.

Refinements are useful because they allow one to manage complexity by setting boundaries. The translation in a refinement deals with some specific property of the structure of the data, and forgets about all other properties which are not relevant. By composing refinements, we are able to modularize our proofs and deal with complexity in an effective manner. This separation of concerns has further advantages. First, it increases our proof's resistance to changes, by restricting the impact of a change to the specific refinements it concerns. Second, it is possible to develop tactics specialized to handle aspects of these refinements, aiding in the automation of the proof and its repairability.

In our project, the first refinement was invoked automatically outside of the theorem prover. When Aeneas translates safe Rust code into Lean, it makes use of Rust's type system to transform the Rust code into a functional representation which removes the need to think about memory and pointers at all. This translation corresponds to the first refinement in our proof of correctness, which is assumed to be correct. All other refinements are performed from within the Lean theorem prover, and correspond to specific types of obstacles that were encountered while performing the proof.

4.1. Eliminating `Aeneas.Std.Result`

When Aeneas translates a Rust definition, the resulting Lean definition operates over the `Aeneas.Std.Result` monad. This monad models the different outcomes that come from calling a Rust function: whether the function panics (represented by `Result.panic`), diverges (represented by `Result.div`), or returns a value instead (represented by `Result.ok`). When programming, the relevant path is that which performs computation, corresponding to the `Result.ok` variant of the monad. One of the objectives of verifying an algorithm is ensuring that, under the conditions where the program will be running on, the program will compute a value instead of throwing an error or looping endlessly. The first refinement in our Lean proofs corresponds to this property, and provides an expression of the program outside of the `Aeneas.Std.Result` monad.

To help reason with `Aeneas.Std.Result`, Aeneas provides the specialized tactic `progress` based on Hoare triples. Hoare triples are the main component used in Hoare logic to describe the effects of the execution of a program. It consists of three components: an operation being applied over some values, a precondition on the state under which this operation was applied and a postcondition expressing what the result of the operation was. With the `progress` attribute, the proof engineer can mark a theorem to be a specification of a particular operation. This theorem must be of the form

$$\forall\ x_1\ \ldots\ x_n\ \exists\ y_1\ \ldots\ y_n,\ \texttt{op}\ \ldots = \texttt{Result.ok}\ y_1\ \ldots\ y_n \wedge \texttt{post}\ \ldots$$

where $\forall\ x_1\ \ldots\ x_n$ are the preconditions, `op …` is the application of the operation and `post …` are the postconditions. When proving a proposition with this same structure, but where the `op` in `op …` can also be a sequence of operations, the user can use the `progress` tactic to find the relevant `progress` theorem for the topmost operation, discharge its preconditions and automatically load the postconditions as local hypothesis in the proof.

To better illustrate the usage of `progress`, we present the following example.

Consider the following proof state on the right. We are analyzing a program whose topmost operation is a call to some `rust.algo`. Calls to `progress` will look for `progress` theorems referencing `rust.algo`.

```
x : U64
⊢ ∃ output,
  (do let y ← rust.algo x
      restOfAlgo y) = Result.ok output
  ∧ postcondition ↑output
```

We have the following `progress` theorem defined for the `rust.algo` function. If we invoke `progress` in a proof state like the previous one, this theorem will be selected to make progress.

```
@[progress] theorem rust.algo.spec
  (x: Aeneas.Std.U64) :
∃ y, rust.algo x = .ok y
   ∧ y.val = spec.algo x.val
```

If we invoke `progress as ⟨y, y_post⟩`, the `progress` tactic will find the previous `rust.algo.spec` theorem, and modify the proof state into the following, where `y` is a new value and `y_post` the postcondition of the `rust.lago.spec` theorem.

```
x y : U64
y_post : ↑y = spec.algo ↑x
⊢ ∃ output,
  restOfAlgo y = Result.ok output
  ∧ postcondition ↑output
```

In the SHA3 project, we used the Aeneas provided `progress` tactic to handle this first type of refinement. In doing so, we prove the termination of our Rust functions under the conditions in which they are called in the SHA3 specification. Furthermore, we developed a `progress*` tactic which attempts to further automate these kinds of arguments. `progress*` will be further discussed in Section 5.1.

4.2. Refinements between equivalent data representations

In the proofs for the SHA3 specification we found multiple instances of elements in the specification being represented by different implementations, which themselves are fully isomorphic. These different representations arise from different considerations made in the SHA3 specification and the Rust implementation. For example, `Array` and `List` are Lean types which can both be used to represent finite ordered enumerations of elements. In particular, `Array` is defined in terms of `List`. Many operations involving `Lists` have an equivalent which handles the `Array` representation, and vice versa, and not all of these operations are simply deferring to their `List` equivalent, such as the case with `Array.foldl` as seen in Figure 3. The reason why Lean has these different representations for the same concept is that, at runtime, `Arrays` have a different implementation. While `Lists` are implemented by linked lists, `Arrays` make use of dynamic arrays. Therefore, the efficiency of certain operations is different between `Lists` and `Arrays`, which makes different implementations emerge depending on which one was used.

```
def List.foldl(f : β → α → β)(init : β)
: List α → β
  | nil      => init
  | cons b l => foldl f (f init b) l
```

```
def Array.foldl(f : β → α → β)(init : β)
    (arr : Array α)(i: Nat := 0): β :=
  if h: i < arr.size then
    foldl f (f init arr[i]) arr (i + 1)
  else init
```

Figure 3: Simplified `List.foldl` (left) and `Array.foldl` (right) definitions.

These sort of situations can be recognized from the existence of a translation function `f` which translates one datatype `A` into a different equivalent representation `B`, and the property that, for any operation `g: A..A → A` there exists an operation `g': B..B → B` such that `f (g a..a') = g' (f a..a')`. We call theorems which state the previous property for some `g` its *interaction theorem*. In the case of `Array` and `List`, the translation function is the `Array.toList` method, and for many operations involving `Lists` Lean has their interaction theorems as part of the standard library (such as `Array.toList_extract`). When these operations were not included in the Lean standard library, we had to include the interaction theorems on our own (such as with `Array.toList_chunks_exact`).

An example of such a situation where the same data had two different representations in the SHA3 verification effort was that of sequences of bits. In the optimized Rust implementation, sequences of bits are packed into sequences of machine integers, while in the specification they are represented as lists of boolean values. The translation between these representations is defined in the `toBits` functions, for both machine integers and sequences of machine integers, with their respective interaction theorems for common operations such as `List`'s `getElem` or `List.set` as seen in Listing 6.

```
theorem List.toBits_set(ls: List (Std.UScalar ty))(u: Std.UScalar ty)(i: Nat)
: (ls.set i u).toBits = ls.toBits.setSlice! (i * ty.numBits) u.toBits
:= by -- ...
```

Listing 6: Interaction theorem of `List.set` with `List.toBits`

In general, these types of refinements are easy to handle using existing Lean automation, such as `simp`, although they do require some effort to define the relevant interaction theorems. However, once defined, the interaction theorems may be reused in different parts of the proofs, reducing their amortized cost when chosen correctly. More so, multiple theorems can be expressed in terms of interaction theorems, since they form the boundary of the layer of abstraction. As long as there is no need to consider the implementation of the translation function, using interaction theorems is enough.

4.3. Separating proofs from values

In theorem provers such as Lean, proofs are values. When choosing the representation of some data structure, the proof engineer must choose where to place the proofs, either with the value data, or as a separate object. For example, one can represent a sequence of `n` elements of type `α` either with a `Vector α n` or with a `List α` with the property that its size is `n`. At first, one may consider that both approaches are equivalent, since they contain the same amount of information. In practice however, we found that separating proofs from other data greatly simplified our proofs.

In the SHA3 specification we make use of dependent types such as `Vector`, for the reasons explained in Section 3. Many of the sequences used in the specification have a fixed length, and therefore theorems on operations involving them needed to state two things: how they affect the elements of the sequence and how they affect its length. Originally I theorized that if we used `Vector` in their representation and relied on theorems about `Vectors` defined in the standard library, I would be able to prove both properties at the same time, making my proofs shorter. For some of the simplest proofs, my hypothesis proved to be correct. However, as proofs got more complicated, some issues started to arise, most of them related to how equality works in Lean.

Equality in Lean is represented by the `Eq` inductive proposition. Some of the common properties of equality are defined in Lean as theorems related to this `Eq` type. For instance, consider Listing 7 for the different definitions of congruence.

```
theorem congrFun{α : Sort u}{β : α → Sort v}{f g : (x : α) → β x}(h : Eq f g)(a : α)
: Eq (f a) (g a)

theorem congrArg{α : Sort u}{β : Sort v}{a b : α}(f : α → β)(h : Eq a b)
: Eq (f a) (f b)
```

Listing 7: Congruence theorems for `Eq`

Theorems such as `congrArg` and `congrFun` are used by tactics such as `rw` to perform rewritings in subterms of the goal expression, since they guarantee that if two expressions are equal, one can be substituted for the other under any context and preserve the equality. However, if we observe the signature of the methods in Listing 7, we observe that `congrFun` is more general than `congrArg`, in that `congrArg` doesn't work for dependent functions. This is because `congrArg` for dependent functions does not hold. If we have `a b : α` and `f: (x : α) → β x`, then even if `a = b`, it is not true that `f a = f b`, since `f a` and `f b` have different types `β a` and `β b`. More precisely, both types are provably equal, but not definitionally.

In practice this means that, when using dependent functions, some rewritings are not type-safe to perform. The current method to perform these kinds of rewritings require using the `cast` function, of signature `Eq α β → α → β`, or other equivalent functions[9]. With it, we can implement a version of `congrArg` which works for dependent functions.

[9]Most dependent types found in the standard library provide their own versions of the `cast` functions, such as `Vector.cast`, `Fin.cast` or `BitVec.cast`. These are specialized to their respective dependent types, and only require a proof of equality of the dependent arguments in the types.

```
theorem congrArgCast{α: Sort u}{β: α → Sort v}{a b: α}(f: (x:α) → β x)(h : Eq a b)
: Eq (f a) (cast (h ▸ rfl) (f b))
:= h ▸ rfl
```

Listing 8: `congrArg` for dependent functions.

Another option is to use a weakened version of equality, `HEq`, which allows one to state equalities when the types of the elements compared are only required to be provably equal. Using `HEq` is equivalent to using `Eq` with `cast`, as seen in Listing 9.

```
theorem Eq_cast_equiv_HEq{α α': Sort u}(a: α)(b: α')(H: α = α')
: Eq a (cast H.symm b) ↔ HEq a b
:= by subst H; simp
```

Listing 9: Equivalence of `Eq` with `cast` and `HEq`

Unfortunately neither of these approaches work well with existing automation tooling. Furthermore, some tactics struggle with dependent types for different reasons, such as the case with `rw` when it tries to derive the motive for a rewriting through unification. For this reason, we decided that the most reliable way to tackle this problem was to avoid dependent types in proofs as much as possible. This prompted us to introduce an intermediate representation of the SHA3 specification which dropped the use of dependent types in favor of simple inductive types and additional lemmas guaranteeing the types' properties.

```
def sponge.absorb(r: Nat )(N: Array Bit)
  (r_bnd: 0 < r ∧ r < b l := by omega)
: Bitstring (b l) :=
  -- [...] Elided
  let mut S := .replicate (b l) false
  for Pᵢ in Ps do
    S := f (S ^^^ (Pᵢ.setWidth (b l)))
  return S
```

```
def IR.absorb.loop (s: List Bool)
    (chunks: List (List Bool))(i: Nat := 0)
: List Bool := Id.run do
  let mut s := s.setWidth (Spec.b 6)
  for Pi in chunks.drop i do
    s := IR.keccak_p (IR.xor s Pi)
  return s

theorem IR.length_absorb_loop (s)(bs)(i)
: (absorb.loop s bs i).length = Spec.b 6
```

Figure 4: Specification of sponge absorb's loop (left) and its intermediate representation (left)

4.4. Other refinements

Not all refinements employed in our proofs corresponded to the previous cases. There are more complex steps in the proofs that are still useful to think about as refinements, for instance when proving that two different implementations of a function are equivalent. This was the case in the SHA3 specification for the `sponge` function, in particular for the `absorb` section of the `sponge` construction during the first stage of the project.

As stated in Section 2, during the first stage of the project we tried to keep the Rust implementation as close as possible to the SHA3 specification. In doing so, we mimicked the specifications' use of lists of bits through slices of booleans in Rust. However, there is a key difference between slices in Rust and the lists used in the specification, which is only relevant for the `sponge.absorb` operation. In the specification, a suffix and some padding is added to the user input. However, in Rust the user input is represented by a slice, which cannot be grown in size. To circumvent this issue, the Rust implementation performs some extra operations which mimic the presence of the suffix and padding at the end of the user input.

When proving the correctness of `sponge.absorb`, the correctness of this transformation needed to be proven as well. This proof was structured in multiple steps.

Finally, refinements can be used to structure different steps of a more complicated proof, for instance when proving equivalent an operation implemented using different primitives, such as linked lists or arrays. This was the case in the SHA3 specification for the `sponge` function, in particular for the `absorb` section of the `sponge` construction. In a first step, a panic-free version of the Rust model was proven equivalent

to another intermediate representation similar to the specification. This constituted the interesting part of the proof, where the different primitives were reconciled. Then, the intermediate representation was proven equivalent to the specification, and in a final theorem composing all steps together, that the panic-free version was equivalent to the Rust model. The interesting part of the proof was further divided in three distinct cases.

5. Exploiting common patterns in refinements

Another advantage of structuring proofs as a composition of refinements is that one can exploit the structure found in these refinements to help with automation of certain tasks. In particular, it is easier to spot common patterns in proofs from a refinement, which allows one to develop specialized tactics to deal with them. In this section, we go over some of the proof automation we used or developed for this project.

5.1. Proofs with `progress`

As we saw in Section 4.1, Aeneas provides a `progress` tactic to aid in proofs involving `Aeneas.Std.Result`. Proofs using `progress` tend to follow quite closely the structure of the program they are proving things about. For example, see Figure 5, where `let*` stands for an alternate syntax for the `progress` tactic.

```
if i < n then do
  let i1 ← r * i
  let i2 ← i + 1#usize
  let i3 ← r * i2
  let chunk ←
    Std.core.slice.index.Slice.index … bs
        { start := i1, end_ := i3 }
  let s1 ← s.xor chunk
  let s2 ← keccak_p s1
  sponge_absorb_initial_loop bs r s2 n i2
else Std.Result.ok s
```

```
split
. let* ⟨ i1, i1_post ⟩ ← Std.Usize.mul_spec
  let* ⟨ i2, i2_post ⟩ ← Std.Usize.add_spec
  let* ⟨ i3, i3_post ⟩ ← Std.Usize.mul_spec
  let* ⟨ chunk, chunk_post ⟩ ← slice_range_spec
  let* ⟨ s1, s1_post ⟩ ← StateArray.xor.spec
  let* ⟨ s2, s2_post ⟩ ← keccak_p.spec
  let* ⟨ res, res_post ⟩ ← spec
  sorry
. sorry
```

Figure 5: Comparison of `progress` proof (right) with the program it works on (left) from `sponge_absorb_initial_loop.spec`

The issue with these kinds of proofs is that they produce a lot of similar and highly structured proof scripts, which are deeply coupled with the code's structure and therefore are sensitive to changes to it. By exploiting this regularity, during my internship I developed a `progress*`[10] tactic which solves these issues. `progress*` traverses the shape of the program being analyzed and calls `progress` on each statement being used, collecting the preconditions which could not be discharged and leaving them as separate goals. Additionally, we also provide a way of giving a proof script to execute when a precondition could not be discharged automatically. We can substitute the whole proof script in Figure 5 with a single invocation of `progress*` to the same effect.

Using `progress*` makes proofs more resistant to changes in the source program, since they become independent of the program structure. For instance, we could change the branches of the `if` statement in the Figure 5 program and `progress*` would still be sufficient to discharge the proof. The only downside is that it is more difficult to debug its behaviour, since it performs many operations at once. To circumvent this issue, we provide an equivalent tactic `progress*?` that provides a code action which replaces the `progress*?` invocation with an equivalent proof script using only `progress` and `split` for bifurcations.

5.2. Automatic dispatch of linear constraints.

When talking about `progress` we mentioned how it will try to automatically dispatch preconditions. This is done by invoking another tactic, `scalar_tac` . `progress` works under the assumption that, in proofs of correctness of cryptographic primitives, most preconditions can be reduced to a linear constraint involving

[10] https://github.com/AeneasVerif/aeneas/pull/451

elements in the local context. The `scalar_tac` tactic solves these kinds of preconditions by preprocessing the current local context with selected theorems, instantiating theorems based on patterns in the goal, and dispatching to a linear constraints solver such as `omega`.

By itself, `omega` is enough to solve the linear constraint. However, it is common for the local context to not contain their hypothesis in the format that `omega` needs them to be to make use of them. To solve this issue, one can mark theorems with the `scalar_tac_simps` attribute to invoke them on the hypothesis before `omega` is called. Furthermore, theorems can be instantiated as local hypothesis to add information relevant to `omega` with the `scalar_tac` attribute, which further requires a pattern to be matched before the selected lemma is included as a local hypothesis. With these additions, `omega` can be extended to handle a large number of preconditions.

```
@[scalar_tac 5 * y + x]
theorem scalar_tac_specific_lemma_for_sha3_keccak_state_packing(x: Fin 5)(y: Fin 5)
: Spec.w 6 * (5 * y + x) ≤ Spec.w 6 * 24
:= Nat.mul_le_mul_left (k := Spec.w 6) <| Nat.le_pred_of_lt (Nat.lt_packing_right x.isLt
y.isLt)
```

Listing 10: Saturation rule for `scalar_tac`. When `5 * y + x` is present in the goal, this lemma is instantiated.

For instance, in the SHA3 project we extended `scalar_tac` to be able to handle `Fin n` types, which were used extensively as indices. Through saturation and rule-based instantiation of lemmas, inequalities with `Fin n` values are transformed into inequalities over natural numbers, replacing `x : Fin n` with `x.val : Nat` and eliminating modulo operations by introducing the hypothesis `m ≤ x ∨ x % m = x` when `x % m` is encountered in the goal. The various extensions made during this project to `scalar_tac` can be found in the `ScalarTac.lean` file in the `sha3.rs` repository.

5.3. Using reflection for precomputed values

Another useful technique in software verification is that of reflection [10]. In the Calculus of Inductive Constructions, when talking about reflection we refer to the relationship between statements made as propositions, and statements made using boolean values. The relevant difference between them is that a proposition does not need to be decidable to be stated, while a boolean expression is by definition the computation of some value, and therefore can be decided simply by allowing the computation to proceed. Therefore, while `Prop` is more flexible for theorem proving, `Bool` is sometimes preferred for its computability. Reflection performs the connection between these two types by introducing a typeclass, which in Lean is called `Decidable`, with the following definition.

```
class inductive Decidable (p : Prop) where
  /-- Proves that `p` is decidable by supplying a proof of `¬p` -/
  | isFalse (h : Not p) : Decidable p
  /-- Proves that `p` is decidable by supplying a proof of `p` -/
  | isTrue (h : p) : Decidable p
```

Listing 11: Definition of `Decidable` in Lean's standard library

When a proposition has an instance of the `Decidable` class, the proof engineer can use tactics such as `decide` or `native_decide` to compute the result of the proposition by switching to its boolean representation. The difference between `decide` and `native_decide` is in how the computation is performed. `decide` uses Lean's kernel, and therefore can be trusted to be correct, at the cost of terrible performance. On the other hand, `native_decide` uses Lean's compilation toolchain, which while it requires the assumption that the compilation is done correctly, gives much better performance. For our project, we considered the benefits given by `native_decide` to outweight its disadvantages.

Reflection is a really useful tool for proofs of cryptographic primitives, in particular when dealing with precomputed values. In the SHA3 project, both ι's `RC` constant and ρ's offsets can be precomputed for an

immediate speedup. Proving that these precomputed values correspond in fact to the values specified in the specification is as simple as asking Lean to check directly using `native_decide` .

```
theorem algos.IOTA_RC.spec
: ∀ iᵣ < 24, IOTA_RC.val[iᵣ]!.toBits = (Spec.Keccak.ι.RC (l := 6) iᵣ).toList
:= by native_decide
```

Listing 12: ι's round constant specification

One particularly useful instance of decidable proposition is that of universal quantifications over a constant-bounded natural value. This allows us to check the validity of a proposition by trying all possible values for the quantified variable. In the case of ρ's offsets computation, this was used

```
theorem algos.RHO_OFFSETS.spec(t: Nat)
: (t_idx: t < 24)
→ let (x,y) := Spec.Keccak.ρ.sequence t
  algos.RHO_OFFSETS.val[x.val]!.val[y.val]!.val = Spec.Keccak.ρ.offset (l:= 6) t
:= by revert t; native_decide
```

Listing 13: ρ offsets' specification. By using `revert` , we quantify over `t`

The ability for a constant-bounded universal quantification to be recognized as decidable comes from definitions found in Lean's `Mathlib` library. However, these are restricted to propositions where the bound on the quantified variable is found as the hypothesis directly after the quantified variable. That is, it works for propositions of the form `∀ (x: Nat), x < 10 → P x → Q x` but not of the form `∀ (x: Nat), P x → x < 10 → Q x`. This restriction became an inconvenience in the project, so we opted to add definitions which allowed the latter family of propositions to be also recognized as decidable. This extension can be found in the `DecideForallLTNested.lean` file, and works by collecting hypotheses which do not correspond to an upper bound of the quantification and pushing them to the postcondition using the property `P → Q ≡ Q ∨ ¬ P`.

```
theorem algos.sha3_512.spec
: ∃ output, algos.sha3_512 bs = .ok output ∧ output.toBits = (Spec.SHA3_512
bs.toBits.toArray).toList
:= by
  simp
  progress with algos.sponge.spec (suffix_len := 2) as ⟨res, res_post⟩
  case suffix_len_spec => simp +decide -- Used here!
  simp +decide [*, Spec.Keccak, Spec.b, Spec.w,
    List.length_toBits, Simp.length_toBits_eq res.val]
  congr
```

Listing 14: Final correctness proof for SHA3-512

References

[1] N. Mouha and C. Celi, "A Vulnerability in Implementations of SHA-3, SHAKE, EdDSA, and Other NIST-Approved Algorithms," in *Topics in Cryptology – CT-RSA 2023*, M. Rosulek, Ed., Cham: Springer International Publishing, 2023, pp. 3–28.

[2] A. Erbsen, J. Philipoom, J. Gross, R. Sloan, and A. Chlipala, "Simple High-Level Code for Cryptographic Arithmetic - With Proofs, Without Compromises," in *2019 IEEE Symposium on Security and Privacy (SP)*, 2019, pp. 1202–1219. doi: 10.1109/SP.2019.00005[11].

[3] M. Polubelova *et al.*, "HACLxN: Verified generic SIMD crypto (for all your favourite platforms)," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 899–918.

[4] J. Protzenko *et al.*, "Verified low-level programming embedded in F*," *Proceedings of the ACM on Programming Languages*, vol. 1, no. ICFP, pp. 1–29, Aug. 2017, doi: 10.1145/3110261[12].

[5] S. Ho and J. Protzenko, "Aeneas: Rust verification by functional translation," *Proceedings of the ACM on Programming Languages*, vol. 6, pp. 711–741, 2022, doi: 10.1145/3547647[13].

[6] L. d. Moura and S. Ullrich, "The Lean 4 Theorem Prover and Programming Language," in *Automated Deduction – CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings*, Berlin, Heidelberg: Springer-Verlag, 2021, pp. 625–635. doi: 10.1007/978-3-030-79876-5_37[14].

[7] T. Coquand and G. Huet, "The calculus of constructions," May 1986. [Online]. Available: https://inria.hal.science/inria-00076024[15]

[8] S. Ullrich and L. de Moura, "'do' unchained: embracing local imperativity in a purely functional language (functional pearl)," *Proc. ACM Program. Lang.*, vol. 6, no. ICFP, Aug. 2022, doi: 10.1145/3547640[16].

[9] "Managing complexity in software engineering," in *Managing Complexity in Software Engineering*, 1990, p. 0.

[10] G. Gonthier, A. Mahboubi, and E. Tassi, "A Small Scale Reflection Extension for the Coq system," 2016. [Online]. Available: https://inria.hal.science/inria-00258384[17]

[11] https://doi.org/10.1109/SP.2019.00005

[12] https://doi.org/10.1145/3110261

[13] https://doi.org/10.1145/3547647

[14] https://doi.org/10.1007/978-3-030-79876-5_37

[15] https://inria.hal.science/inria-00076024

[16] https://doi.org/10.1145/3547640

[17] https://inria.hal.science/inria-00258384